

Universidad de las Fuerzas Armadas ESPE

Sede Sangolquí

Carrera: Ingeniería de Software

NRC: 36873

Tarea 2.1: Resolución de problemas – Creación y ejecución de pantallas (Activity) en Flutter para los ejercicios 6 al 10

Integrantes:

Marcelo Acuña

Abner Arboleda

Christian Bonifaz

Esteban Santos

Materia: Desarrollo de Aplicaciones Móviles

Docente: Ing. Doris Chicaiza .Mgtr

Sangolquí, 13 de octubre de 2026

Índice

1. Introducción **2**

2. Objetivos **2**

2.1. Objetivo general 2

2.2. Objetivos específicos 2

3. Trabajos relacionados **3**

4. Desarrollo **3**

4.1. Conceptos básicos 3

4.2. Herramientas 4

4.2.1. Android Studio 4

4.2.2. Flutter y Dart 4

4.2.3. Visual Studio Code 4

4.3. Procedimiento de desarrollo 4

4.4. Problemas encontrados y soluciones 21

5. Conclusiones **21**

6. Recomendaciones **22**

Anexos **22**

1. Introducción

En Android, cada pantalla con la que interactúa el usuario se asocia tradicionalmente a una *Activity*. Flutter adopta un enfoque distinto: la aplicación se ejecuta dentro de una única `MainActivity` que aloja el motor de Flutter, y cada pantalla se construye como un *widget* al que se llega mediante el sistema de navegación y rutas del *framework* (Flutter Team, s.f.-a,s). Entender esta diferencia es necesario para crear aplicaciones con varias pantallas, pasar de una a otra y regresar al menú sin perder la organización del código.

Este informe documenta el desarrollo de una aplicación móvil en Flutter y Dart que resuelve los ejercicios 6 al 10 asignados al NRC 36873: impuesto vehicular, cálculo de saldos del banco "Bandido de peluche", ahorro anual, juego de adivinar un número e intervalo con sumatoria. La aplicación tiene un menú principal desde el que se accede a cada ejercicio en una pantalla independiente; la lógica de cada problema está separada de la interfaz, y los componentes visuales se organizan siguiendo la metodología *Atomic Design* (Frost, 2016). El informe describe la estructura del proyecto, el análisis y la implementación de cada ejercicio, y los problemas encontrados con sus soluciones.

2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación móvil en Flutter y Dart que resuelva los ejercicios 6 al 10 del NRC 36873, mediante un menú principal con rutas nombradas, lógica separada en clases de Dart independientes de la interfaz y componentes visuales organizados con *Atomic Design*, para aplicar la creación, ejecución y navegación entre pantallas de una aplicación móvil.

2.2. Objetivos específicos

- Organizar el proyecto en carpetas que separen la lógica, el tema visual, las rutas y las vistas.
- Implementar la lógica de cada ejercicio en una clase de Dart pura que valide los datos de entrada.
- Construir un menú principal que navegue a cada ejercicio con rutas nombradas y permita regresar.
- Diseñar para cada ejercicio una pantalla con formularios, mensajes de validación y tablas de resultados.
- Ejecutar la aplicación en un emulador Android y comprobar el funcionamiento de cada ejercicio.

3. Trabajos relacionados

1. Quisaguano et al. (2022) comparan Flutter, Xamarin y Android Studio como entornos de desarrollo móvil y describen la preparación de un proyecto Flutter en Windows con Visual Studio Code. Es el antecedente directo del entorno usado en este trabajo, que reutiliza la configuración del laboratorio anterior.

2. Fentaw (2020) compara React Native y Flutter construyendo la misma aplicación con varias pantallas y consumo de servicios. Su análisis de la estructura de ambas aplicaciones respalda el uso de Flutter para aplicaciones con navegación entre vistas, como la desarrollada aquí.

3. George et al. (2025) contrastan Android nativo, Flutter y React Native mediante una aplicación de control de gastos, un dominio de cálculos monetarios similar a los ejercicios 6, 7 y 8. Concluyen que Flutter ofrece buena productividad sin sacrificar la calidad del código.

4. Torres Peña y Andrade Solórzano (2021) desarrollaron con Flutter una aplicación informativa y de apoyo académico para la Universidad Politécnica Salesiana, con un menú principal que da acceso a distintos módulos. Su organización por módulos se asemeja al menú de ejercicios de esta aplicación.

5. Safavi (2025) evalúa el rendimiento de React Native y Flutter y construye con Flutter un prototipo de gestión de bodega basado en formularios y listas. Sus resultados muestran que la compilación anticipada y el motor de renderizado propio de Flutter mantienen fluidas las interfaces con muchos campos, como los formularios dinámicos de los ejercicios 6 y 7.

4. Desarrollo

4.1. Conceptos básicos

- ***Activity*:** componente de Android que representa una pantalla con la que el usuario interactúa. En Flutter existe una sola `MainActivity`, que aloja el motor de Flutter; las demás pantallas son *widgets*.

- ***Widget*:** unidad básica de la interfaz en Flutter; toda pantalla, botón o texto es un *widget* (Flutter Team, s.f.-a).

- ***StatelessWidget* y *StatefulWidget*:** el primero no cambia después de construirse; el segundo guarda un estado y se vuelve a dibujar cuando se llama a `setState()` (Flutter Team, s.f.-c).

- **`TextEditingController`:** objeto que lee y modifica el texto de un campo de entrada. Debe liberarse con `dispose()` cuando la pantalla se destruye.
- **Rutas nombradas:** cadenas como `/ej06` asociadas a una pantalla. `Navigator.pushNamed` abre la pantalla y el botón atrás la cierra (Flutter Team, s.f.-b).
- ***Atomic Design*:** metodología que organiza la interfaz en átomos, moléculas, organismos, plantillas y páginas, de lo más simple a lo más complejo (Frost, 2016).
- **Valor centinela:** dato especial que indica el fin de una entrada; en el ejercicio 10 es el número 0.

4.2. Herramientas

4.2.1. Android Studio

Proporciona el SDK de Android, las herramientas de compilación y el emulador en el que se ejecutó la aplicación (Google, s.f.). Se usó la misma instalación configurada en el laboratorio 1.1.

4.2.2. Flutter y Dart

Se utilizó Flutter 3.47.7 (canal *stable*) con Dart 3.13.5. Flutter aporta los *widgets* de Material Design, el sistema de navegación y el paquete `flutter_test`; Dart es el lenguaje en el que se escribieron la lógica y las pantallas (Flutter Team, s.f.-a; Dart Team, s.f.).

4.2.3. Visual Studio Code

Editor principal del proyecto, con las extensiones de Flutter y Dart para autocompletado, ejecución en el emulador y *hot reload* (Microsoft, s.f.).

4.3. Procedimiento de desarrollo

Paso 1: Crear el proyecto

Se creó el proyecto con un identificador de organización propio del NRC:

```
flutter create --org com.nrc36873 tarea2_1
cd tarea2_1
```

Flutter generó la carpeta `android/` con la única *Activity* de la aplicación, ubicada en `android/app/src/main/kotlin/com/nrc36873/tarea2_1/MainActivity.kt`. Todo el código propio se escribió dentro de `lib/`.

Paso 2: Organizar la estructura de carpetas

El código se dividió para que cada carpeta tenga una sola responsabilidad. La lógica no importa Flutter, por lo que puede probarse sin interfaz; las vistas se agrupan por nivel de *Atomic Design*.

Código 1. Estructura de carpetas del proyecto.

```
tarea2_1/
|-- android/                        Proyecto nativo (MainActivity.kt)
|-- lib/
|   |-- main.dart                   Punto de entrada: MaterialApp,
   tema y rutas
|   |-- rutas.dart                  Registro de ejercicios y mapa de
   rutas
|   |-- logica/                     Clases Dart puras, sin Flutter
|   |   |-- impuesto_logica.dart        Ejercicio 6
|   |   |-- banco_logica.dart           Ejercicio 7
|   |   |-- ahorro_logica.dart          Ejercicio 8
|   |   |-- adivina_logica.dart         Ejercicio 9
|   |   `-- intervalo_logica.dart       Ejercicio 10
|   |-- tema/
|   |   `-- tema_app.dart           Paleta de colores y ThemeData
|   `-- views/
|       |-- atoms/                  Piezas mínimas (tarjeta, ícono,
   insignia...)
|       |-- molecules/              Combinaciones de átomos (aviso,
   fila de dato...)
|       |-- organisms/              Secciones completas (encabezado,
   tabla...)
|       |-- templates/              Estructura común de cada pantalla
|       `-- pages/                  Menú principal y una pantalla por
   ejercicio
|-- test/
|   |-- logica_test.dart            Pruebas unitarias de la lógica
|   `-- widget_test.dart            Prueba de navegación del menú
`-- pubspec.yaml
```

Cada carpeta de `views/` tiene un archivo índice (por ejemplo `atoms/atoms.dart`) que exporta sus componentes, de modo que una pantalla los importa con una sola línea. La Tabla 1 resume los componentes de cada nivel.

Cuadro 1. Componentes de la interfaz por nivel de *Atomic Design*.

Nivel	Componentes	Uso
Átomos	`TarjetaTravo`, `IconoPastel`, `InsigniaNumero`, `EtiquetaEstado`, `TituloSeccion`	Contenedores, íconos, números de fila, etiquetas Sí/No y títulos.
Moléculas	`AvisoTravo`, `FilaDato`, `FilaRegla`, `IndicadorPaso`, `TarjetaTasa`, `TotalDestacado`	Mensajes de error, filas de resultados, reglas de cálculo, pasos y totales.
Organismos	`EncabezadoTravo`, `TablaTravo`, `TarjetaEjercicio`	Encabezado con botón atrás, tabla con desplazamiento horizontal y tarjetas del menú.
Plantillas	`PlantillaPantalla`	Encabezado y contenido desplazable comunes a todas las pantallas.
Páginas	`MenuPrincipal` y las cinco pantallas de ejercicios	Pantallas completas a las que apuntan las rutas.

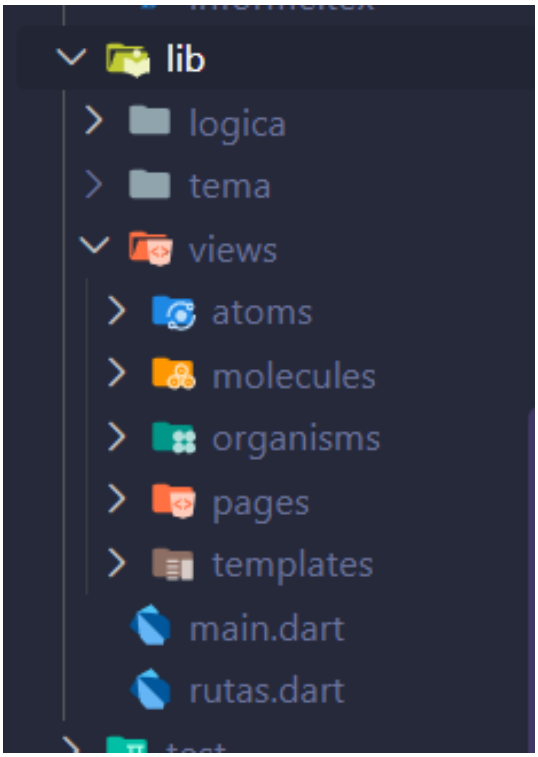


Figura 1. Estructura del proyecto en Visual Studio Code.

Paso 3: Definir el tema visual

En `lib/tema/tema_app.dart` se definió la clase `TravoColores` con la paleta de la aplicación (morado principal, naranja, rosa, turquesa y verde) y la función `temaTravo()`,

que configura Material 3, los botones con forma de píldora y los campos de texto redondeados. Así todas las pantallas comparten el mismo aspecto sin repetir estilos.

Paso 4: Registrar las rutas y construir el menú

En `lib/rutas.dart` se creó una lista de registro con los datos de cada ejercicio. El menú y el mapa de rutas se generan a partir de esa lista, por lo que agregar un ejercicio solo requiere agregar una línea.

Código 2. Registro de ejercicios y rutas nombradas (fragmento de `rutas.dart`).

```
final List<EjercicioInfo> ejercicios = [
  EjercicioInfo(id: '6', titulo: 'Impuesto vehicular', descripcion: '
   Impuesto por vehículo, por categoría y total', ruta: '/ej06',
   pantalla: (_) => const ImpuestoPantalla()),
  EjercicioInfo(id: '7', titulo: 'Banco Bandido de peluche', ...,
   ruta: '/ej07', ...),
  EjercicioInfo(id: '8', titulo: 'Ahorro anual', ..., ruta: '/ej08',
   ...),
  EjercicioInfo(id: '9', titulo: 'Adivina el número', ..., ruta: '/
   ej09', ...),
  EjercicioInfo(id: '10', titulo: 'Intervalo y sumatoria', ..., ruta:
    '/ej10', ...),
];

/// Rutas nombradas de la aplicación.
final Map<String, WidgetBuilder> rutas = {
  for (final e in ejercicios) e.ruta: e.pantalla,
};
```

El archivo `main.dart` solo crea el `MaterialApp` con el tema, el menú como pantalla inicial y el mapa de rutas:

Código 3. Punto de entrada de la aplicación (`main.dart`).

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Ejercicios NRC 36873',
      theme: temaTravo(),
      home: const MenuPrincipal(),
```

```
      routes: rutas,
    );
  }
}
```

Cada tarjeta del menú llama a `Navigator.pushNamed(context, e.ruta)`. El encabezado de las pantallas muestra un botón atrás solo cuando existe una pantalla anterior (`Navigator.of(context).canPop()`), con lo que se regresa al menú.



Figura 2. Menú principal de la aplicación.

Todas las pantallas de ejercicios siguen el mismo patrón: un `StatefulWidget` con un `TextEditingController` por campo, liberado en `dispose()`; un método por acción (`generar`, `calcular`, `limpiar`) que lee los campos, llama a la clase de lógica y actualiza la vista con `setState()`; y un botón flotante para limpiar el formulario.

Paso 5: Ejercicio 6 – Impuesto vehicular

Enunciado. El gerente de una compañía automotriz desea determinar el impuesto que pagará por cada automóvil, el total por cada categoría y el total de todos los vehículos. Los vehículos con clave 1 pagan el 10 % de su valor, los de clave 2 el 7 % y los de clave 3 el 5 %.

Análisis.

- *Entradas:* número de vehículos N y, por cada uno, su valor y su clave.

- *Proceso:* $impuesto = valor \times tasa(clave)$; el impuesto se acumula en el total de su clave y en el total general.
- *Salidas:* impuesto por vehículo, total por clave y total general.
- *Validación:* clave distinta de 1, 2 o 3, valor negativo o campo vacío.

Código 4. Lógica del ejercicio 6 (`impuesto_logica.dart`).

```dart
class ImpuestoLogica {
  static const Map<int, double> tasas = {1: 0.10, 2: 0.07, 3: 0.05};

  // Acumuladores por clave y total general
  final Map<int, double> totalPorClave = {1: 0.0, 2: 0.0, 3: 0.0};
  double totalGeneral = 0.0;

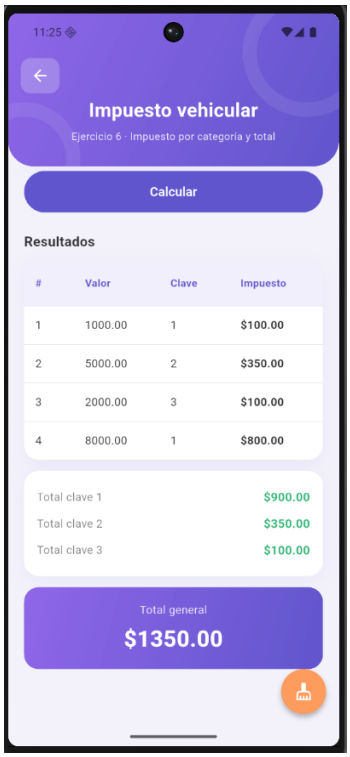
  double impuesto(double valor, int clave) {
    final tasa = tasas[clave];
    if (tasa == null) {
      throw ArgumentError('Clave inválida: $clave (debe ser 1, 2 o 3)
   ');
    }
    if (valor < 0) {
      throw ArgumentError('El valor no puede ser negativo');
    }
    return valor * tasa;
  }

  /// Calcula el impuesto, lo suma a los acumuladores y lo devuelve.
  double registrar(double valor, int clave) {
    final imp = impuesto(valor, clave);
    totalPorClave[clave] = totalPorClave[clave]! + imp;
    totalGeneral += imp;
    return imp;
  }
}
```

Pantalla. Muestra las tres tasas en tarjetas de color, un campo para N (entre 1 y 50) y el botón *Generar formulario*, que crea una fila con valor y clave por vehículo. Al presionar *Calcular* aparece una tabla con el impuesto de cada vehículo, los totales por clave y el total general. Si una fila tiene datos inválidos, el error se muestra en esa fila y los demás vehículos se siguen calculando.



(a) Formulario de vehículos



(b) Resultados

Figura 3. Ejercicio 6: impuesto vehicular.



Figura 4. Validación de una clave inválida en el ejercicio 6.

Paso 6: Ejercicio 7 – Banco "Bandido de peluche"

Enunciado. El banco desea calcular, para cada uno de sus N clientes, el saldo actual, el pago mínimo y el pago para no generar intereses, además del monto ganado por intereses con los clientes morosos. De cada cliente se conoce el saldo anterior, el monto de las compras y el pago depositado en el corte anterior. El pago mínimo es el 15 % del saldo actual y el pago para no generar intereses es el 85 %; el saldo actual debe incluir el 12 % de intereses y $200 de multa por no haber realizado el pago mínimo.

Supuestos. El enunciado no indica cuál era el pago mínimo del corte anterior, por lo que se adoptaron los siguientes criterios, documentados también en el código:

1. El pago mínimo de referencia es el 15 % del saldo anterior.
2. Un cliente es moroso si su pago depositado es menor que ese pago mínimo.
3. $base = saldoAnterior - pago + compras$.
4. Si es moroso: $interes = 0.12 \times base$ y $multa = 200$; si no, ambos son 0.
5. $saldoActual = base + interes + multa$.
6. Si el cliente pagó más de lo que debía, el saldo queda a favor (negativo) y sus pagos mínimo y sin intereses son 0.
7. El total ganado por intereses suma solo el interés de los morosos.

Pseudocódigo.

Código 5. Pseudocódigo del ejercicio 7.

```
Algoritmo BancoBandidoDePeluche
  Constantes
    TASA_MIN <- 0.15     TASA_SIN <- 0.85
    TASA_INT <- 0.12     MULTA    <- 200
  Variables
    N, i: entero
    saldoAnt, compras, pago, pmAnt, base, interes, multa: real
    saldoAct, saldoPagar, pagoMin, pagoSin, totalIntereses: real
Inicio
  Leer N
  totalIntereses <- 0
  Para i <- 1 hasta N hacer
    Leer saldoAnt, compras, pago
    pmAnt <- TASA_MIN * saldoAnt
    base  <- saldoAnt - pago + compras
    Si pago < pmAnt entonces
```

```
      interes <- TASA_INT * base
      multa   <- MULTA
      totalIntereses <- totalIntereses + interes
    Sino
      interes <- 0
      multa   <- 0
    FinSi
    saldoAct <- base + interes + multa
    Si saldoAct > 0 entonces
      saldoPagar <- saldoAct
    Sino
      saldoPagar <- 0
    FinSi
    pagoMin <- TASA_MIN * saldoPagar
    pagoSin <- TASA_SIN * saldoPagar
    Escribir "Cliente", i, saldoAct, pagoMin, pagoSin
  FinPara
  Escribir "Intereses ganados con morosos:", totalIntereses
Fin
```

Diagrama de flujo. La Figura 5 representa el mismo algoritmo.

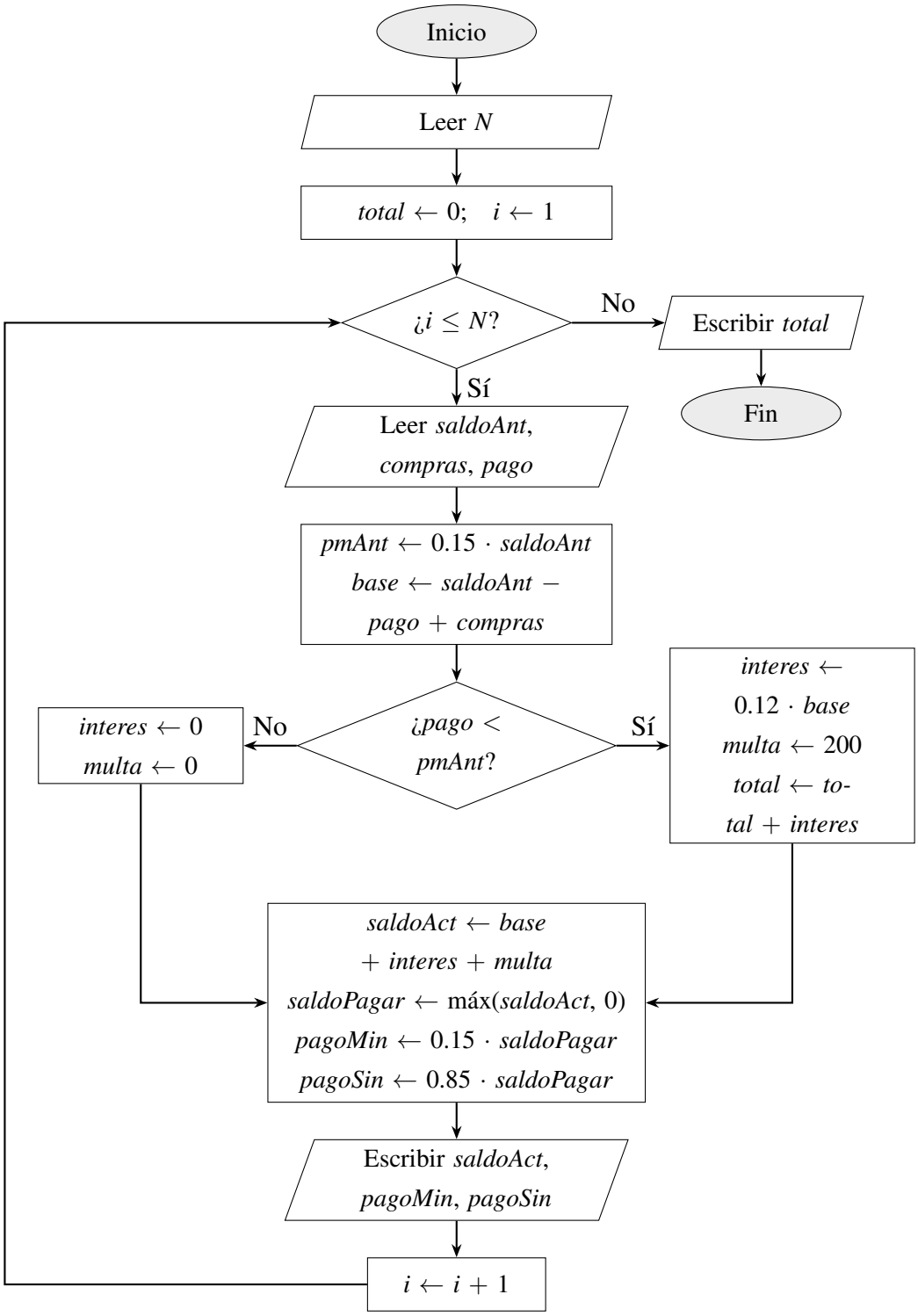


Figura 5. Diagrama de flujo del ejercicio 7.

Código 6. Cálculo de un cliente en el ejercicio 7 (`banco_logica.dart`).

```
ResultadoCliente calcular(Cliente c) {
  if (c.saldoAnterior < 0 || c.compras < 0 || c.pagoDepositado < 0) {
    throw ArgumentError('Los datos del cliente no pueden ser
   negativos');
```

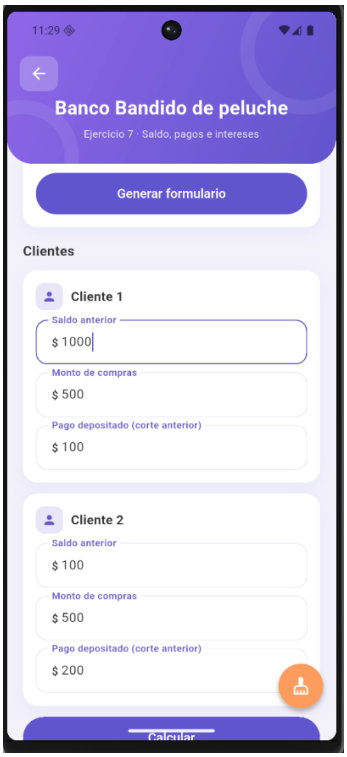
```
  }

  final pagoMinimoAnterior = tasaPagoMinimo * c.saldoAnterior; //
   paso 1
  final moroso = c.pagoDepositado < pagoMinimoAnterior; // paso 2
  final base = c.saldoAnterior - c.pagoDepositado + c.compras; //
   paso 3
  final interes = moroso ? tasaInteres * base : 0.0; // paso 4
  final multa = moroso ? multaMorosidad : 0.0;
  final saldoActual = base + interes + multa; // paso 5

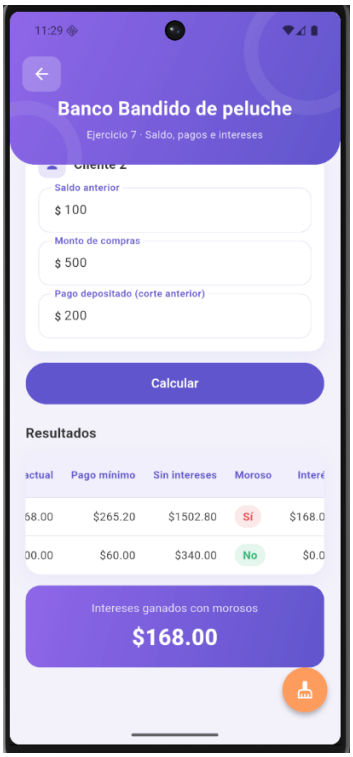
  // Si el cliente pagó de más, el saldo queda a favor (negativo) y
   no debe nada.
  final saldoAPagar = saldoActual > 0 ? saldoActual : 0.0;

  return ResultadoCliente(
    saldoActual: saldoActual,
    pagoMinimo: tasaPagoMinimo * saldoAPagar, // paso 6
    pagoSinIntereses: tasaSinIntereses * saldoAPagar, // paso 7
    moroso: moroso,
    interes: interes,
    multa: multa,
  );
}
```

Pantalla. Presenta las reglas del banco, un campo para N y un formulario por cliente con saldo anterior, monto de compras y pago depositado. La tabla de resultados muestra saldo actual, pago mínimo, pago sin intereses, una etiqueta roja o verde que indica si el cliente es moroso y el interés cobrado; al final se destaca el total de intereses ganados con los morosos.



(a) Formulario de clientes



(b) Resultados

Figura 6. Ejercicio 7: banco “Bandido de peluche”.

Paso 7: Ejercicio 8 – Ahorro anual

Enunciado. Determinar cuánto ahorrará una persona en un año si al final de cada mes deposita cantidades variables, y cuánto lleva ahorrado cada mes.

Análisis. Las entradas son los doce depósitos mensuales. En cada mes el saldo acumulado es el saldo anterior más el depósito del mes; las salidas son el saldo acumulado de cada mes y el total del año. No se admiten depósitos negativos.

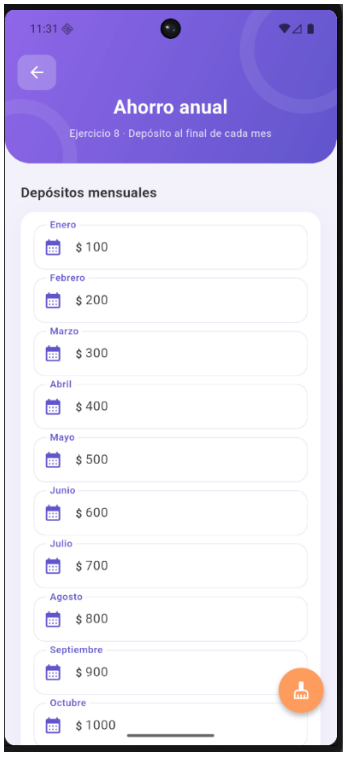
Código 7. Lógica del ejercicio 8 (`ahorro_logica.dart`).

```
ResultadoAhorro calcular(List<double> depositos) {
  if (depositos.length != 12) {
    throw ArgumentError('Se requieren 12 depósitos');
  }
  if (depositos.any((d) => d < 0)) {
    throw ArgumentError('Los depósitos no pueden ser negativos');
  }

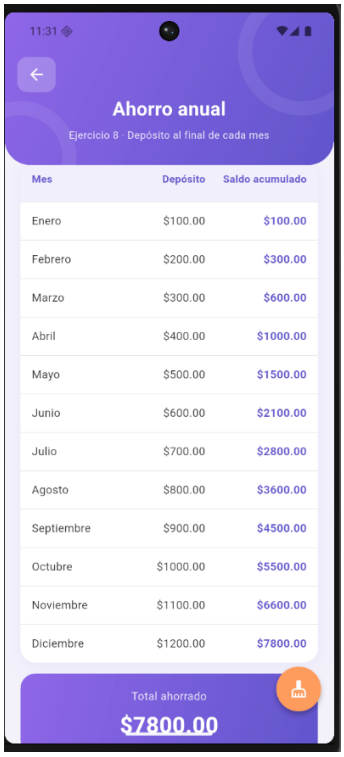
  final acumulados = <double>[];
  double saldo = 0.0;
  for (final d in depositos) {
    saldo += d;
```

```
    acumulados.add(saldo);
  }
  return ResultadoAhorro(acumulados, saldo);
}
```

Pantalla. Contiene doce campos etiquetados con el nombre de cada mes y el botón *Calcular*. El resultado es una tabla con el mes, el depósito y el saldo acumulado, seguida del total ahorrado.



(a) Depósitos mensuales



(b) Saldo acumulado y total

Figura 7. Ejercicio 8: ahorro anual.

Paso 8: Ejercicio 9 – Adivina el número

Enunciado. La aplicación genera un número aleatorio del 1 al 100 y el usuario tiene 10 intentos para adivinarlo. Tras cada intento indica si el número secreto es mayor o menor y cuántos intentos quedan. Si el usuario acierta, se informa en cuántos intentos lo logró; si agota los intentos, se muestra el número secreto.

Análisis. El estado del juego (número secreto, intentos usados y si terminó) se guarda en la clase `AdivinaLogica`. Cada intento devuelve uno de cuatro resultados: `mayor`, `menor`, `acierto` o `sinIntentos`. Un número fuera del rango 1–100 se rechaza sin consumir intento. El constructor acepta un número secreto fijo para poder probar la lógica.

Código 8. Procesamiento de un intento en el ejercicio 9 (`adivina_logica.dart`).

```
AdivinaLogica({Random? random, int? secreto})
```

```
    : numeroSecreto = secreto ?? (random ?? Random()).nextInt(maximo)
    + minimo;

int get intentosRestantes => maxIntentos - intentosUsados;

ResultadoIntento intentar(int n) {
  if (terminado) throw StateError('El juego ya terminó');
  if (n < minimo || n > maximo) {
    throw ArgumentError('El número debe estar entre $minimo y $maximo
   ');
  }

  intentosUsados++;
  if (n == numeroSecreto) {
    terminado = true;
    return ResultadoIntento.acierto;
  }
  if (intentosUsados >= maxIntentos) {
    terminado = true;
    return ResultadoIntento.sinIntentos;
  }
  return n < numeroSecreto ? ResultadoIntento.mayor :
   ResultadoIntento.menor;
}
```

Pantalla. Una tarjeta muestra “Te quedan X intentos”. El campo solo acepta dígitos (máximo tres) y, al presionar *Probar*, aparece la pista en un aviso naranja. Al terminar se muestra un recuadro verde con el número de intentos usados o uno rojo con el número secreto; debajo se listan los números intentados y el botón *Nuevo juego* reinicia la partida.



(a) Pista e intentos restantes



(b) Acierto

Figura 8. Ejercicio 9: adivina el número.



Figura 9. Ejercicio 9: fin del juego al agotar los intentos.

Paso 9: Ejercicio 10 – Intervalo y sumatoria

Enunciado. Pedir el límite inferior y el superior de un intervalo, volviendo a pedirlos si el inferior es mayor que el superior. Luego se ingresan números hasta introducir 0, y se informa la suma de los números dentro del intervalo abierto, cuántos números están fuera de él y si se introdujo algún número igual a uno de los límites.

Análisis. Un número x está dentro del intervalo abierto si $lo < x < hi$, y fuera si $x < lo$ o $x > hi$. Un número igual a un límite no se suma ni se cuenta como fuera, pero activa el indicador de igualdad. El 0 se usa como valor centinela: termina la entrada y no se cuenta como dato.

Código 9. Análisis de los números en el ejercicio 10 (`intervalo_logica.dart`).

```
ResultadoIntervalo analizar(double lo, double hi, List<double>
   numeros) {
  final error = validarLimites(lo, hi);
  if (error != null) throw ArgumentError(error);

  double suma = 0.0;
  int fuera = 0;
  bool igualLimite = false;

  for (final x in numeros) {
    if (x > lo && x < hi) {
      suma += x;
    } else if (x < lo || x > hi) {
      fuera++;
    } else {
      // x == lo o x == hi: no se suma ni cuenta como fuera
      igualLimite = true;
    }
  }
  return ResultadoIntervalo(suma, fuera, igualLimite);
}
```

Pantalla. Se organiza en dos pasos numerados. En el paso 1 se ingresan los límites; si el inferior es mayor, aparece un error y el usuario permanece en ese paso. En el paso 2 se agregan los números uno por uno y se muestran como chips; al ingresar 0 aparecen la suma dentro del intervalo, la cantidad de números fuera y el mensaje sobre los límites.



(a) Validación de límites



(b) Resultados

Figura 10. Ejercicio 10: intervalo y sumatoria.

4.4. Problemas encontrados y soluciones

Problema	**Causa**	**Solución**
Criterio de morosidad del ejercicio 7	El enunciado no indica el pago mínimo del corte anterior.	Se tomó el 15 % del saldo anterior como referencia y el supuesto se documentó en el código y en este informe.
Pagos negativos en el ejercicio 7	Si el pago supera la deuda, el saldo actual queda negativo.	Los pagos mínimo y sin intereses se calculan sobre $\max(saldoActual, 0)$.
Cantidad variable de campos	Los ejercicios 6 y 7 dependen de N, que el usuario define en tiempo de ejecución.	Los controladores se crean con `List.generate`, se liberan con `dispose()` al regenerar o salir, y N se limita a 50.
Un dato inválido detenía todo el cálculo	La lógica lanza `ArgumentError` ante datos inválidos.	La pantalla captura el error por fila, lo muestra en esa fila y continúa con los demás registros.
El 0 del ejercicio 10 podía contarse como dato	El 0 marca el fin de la entrada.	Se trata como centinela: dispara el análisis y nunca se agrega a la lista.
Intentos inválidos en el ejercicio 9	Un número fuera de 1–100 no debe gastar un intento.	El campo acepta solo dígitos y la lógica rechaza el número antes de incrementar el contador.
Tablas más anchas que la pantalla	El ejercicio 7 tiene seis columnas de resultados.	El organismo `TablaTravo` envuelve la tabla en un desplazamiento horizontal.

5. Conclusiones

1. En Flutter una sola `MainActivity` aloja toda la aplicación y cada pantalla es un *widget* al que se llega por una ruta nombrada. Generar el menú y el mapa de rutas desde una misma lista de registro permitió acceder a los cinco ejercicios y regresar al menú sin duplicar código.

2. Separar la lógica en clases de Dart sin dependencias de Flutter dejó a cada pantalla la tarea de leer los campos, llamar a la lógica y mostrar los resultados, y concentró en un solo lugar las fórmulas y los supuestos, como el criterio de morosidad del ejercicio 7, lo que facilita revisarlos y modificarlos.

3. Organizar la interfaz con *Atomic Design* hizo que las cinco pantallas compartieran tarjetas, avisos, tablas y encabezados, lo que dio un aspecto uniforme a la aplicación y redujo cada página a la composición de componentes ya probados.

6. Recomendaciones

1. Confirmar con el docente los enunciados ambiguos, como el criterio de morosidad del ejercicio 7, antes de implementarlos, y documentar en el código cualquier supuesto adoptado.

2. Mantener la lógica de cada ejercicio en su propia clase, separada de la pantalla, para poder corregir una fórmula sin modificar la interfaz y reutilizar los cálculos en otras pantallas.

3. Liberar siempre los `TextEditingController` en `dispose()`, sobre todo cuando se crean dinámicamente, y limitar la cantidad de campos que el usuario puede generar para evitar consumo innecesario de memoria.

Anexos

Anexo A: Repositorio del proyecto

El código fuente completo de la aplicación está publicado en GitHub:

https://github.com/adarboleda/dispositivos_moviles

Referencias

Dart Team. (s.f.). *Introduction to Dart*. Google. https://dart.dev/language

Fentaw, A. E. (2020). *Cross platform mobile application development: A comparison study of React Native vs Flutter* [Tesis de maestría, University of Jyväskylä]. JYX. https://urn.fi/URN:NBN:fi:jyu-202006295155

Flutter Team. (s.f.-a). *Flutter documentation*. Google. https://docs.flutter.dev

Flutter Team. (s.f.-b). *Navigation and routing*. Google. https://docs.flutter.dev/ui/navigation

Flutter Team. (s.f.-c). *Adding interactivity to your Flutter app*. Google. https://docs.flutter.dev/ui/interactivity

Frost, B. (2016). *Atomic design*. Brad Frost. https://atomicdesign.bradfrost.com

George, K., Kirubanand, V. B., y Sandeep, J. (2025). Cross-platform vs. native mobile development: A comprehensive study using an expense tracker application. En *Proceedings of the 2025 International Conference on Computational Innovations and Sustainable Technologies (ICCIST 2025)*. IEEE.

Google. (s.f.). *Android Studio*. Android Developers. https://developer.android.com/studio

Microsoft. (s.f.). *Visual Studio Code*. https://code.visualstudio.com

Quisaguano, L. R., Camalle, T. G., y Toca, J. W. (2022). Análisis comparativo de entornos de desarrollo móvil. *Ciencia Latina Revista Científica Multidisciplinar, 6*(4), 4478–4498. https://doi.org/10.37811/cl_rcm.v6i4.2950

Safavi, S. (2025). *Performance evaluation of cross-platform mobile frameworks: A comparative study of React Native and Flutter* [Tesis, Università degli Studi di Padova]. Padua Thesis Archive. https://hdl.handle.net/20.500.12608/87277

Torres Peña, P. A., y Andrade Solórzano, B. A. (2021). *Aplicación móvil informativa y de apoyo académico de la Universidad Politécnica Salesiana* [Trabajo de titulación de grado, Universidad Politécnica Salesiana]. Repositorio Institucional UPS. https://dspace.ups.edu.ec/handle/123456789/21490